

CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

UniswapV2Router02

0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D

Ethereum

Compiler: v0.6.6+commit.6c089d02

Date: October 7, 2026

OVERALL RISK RATING AI-assisted audit. Not a substitute for manual review.

Low

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. This is the canonical, immutable UniswapV2Router02 contract that has processed billions of dollars in volume since 2020 without a critical exploit attributable to router logic. The contract has no owner-gated functions, no proxy/upgrade pattern despite the 'upgradeable' flag in the request metadata, and correctly uses Safe-Math, checked external-call return values via TransferHelper, and a deadline guard. Findings are limited to minor code-quality issues (use of `assert` for external-call validation) and inherent, well-documented design trade-offs (MEV/sandwich exposure via user-supplied slippage parameters, and the need to use the correct fee-on-transfer-supporting function variants). No critical or high-severity issues were identified in the router logic itself.

Findings (4)

F-001 · SWC-110

Low

Use of assert() to validate external call return value instead of require()

UniswapV2Router02.sol:185–186

DESCRIPTION

Several functions use `assert(IWETH(WETH).transfer(...))` to validate the return value of an external call to the WETH contract. `assert` is intended to check for internal invariants that should never be false; in Solidity ^0.6.x a failed assert consumes all remaining gas (pre-Byzantium semantics still applied via `INVALID` opcode under the EVM used at the time, and even post-London it still signals a different class of failure than a normal revert). Using it to gate an external dependency's return value is a validation check, not an invariant, and is more appropriately expressed with `require`.

IMPACT

If the canonical WETH9 contract's `transfer` were ever to return `false` (it does not under current WETH9 bytecode, which always returns true or reverts), the resulting failure mode would burn all remaining gas for the caller instead of reverting cleanly with a gas refund, degrading UX and complicating off-chain error decoding. This is a defense-in-depth / best-practice issue rather than an exploitable vulnerability given WETH9's current immutable bytecode.

RECOMMENDATION

Replace `assert(...)` with `require(..., "UniswapV2Router: WETH\_TRANSFER\_FAILED")` for all WETH transfer checks (addLiquidityETH, swapExactETHForTokens, swapETHForExactTokens, swapExactETHForTokensSupportingFeeOnTransferTokens).

```
IWETH(WETH).deposit{value: amountETH}();
assert(IWETH(WETH).transfer(pair, amountETH));
liquidity = IUniswapV2Pair(pair).mint(to);
```

F-002 · SWC-114

Low

Swap functions are exposed to front-running / sandwich attacks when callers supply loose or zero slippage bounds

UniswapV2Router02.sol:293–300

DESCRIPTION

All swap entry points (`swapExactTokensForTokens`, `swapExactETHForTokens`, etc.) compute expected output/input amounts from on-chain reserves at execution time and only protect the caller via the externally supplied `amountOutMin` / `amountInMax` parameters. The router itself performs no additional protection (e.g., TWAP sanity check, max price-impact cap). If an integrating wallet or dApp sets `amountOutMin = 0` or an overly permissive bound, or if a transaction sits in the public mempool, MEV searchers can sandwich the trade for profit at the user's expense.

IMPACT

End users who set loose slippage tolerances (or integrators who default to 0) can lose significant value to sandwich bots. This is a long-standing, well-known characteristic of all constant-product AMM routers, not a bug unique to this deployment, but it is the single most exploited real-world attack vector against Uniswap v2 traders.

RECOMMENDATION

This is inherent to the AMM design and cannot be fully solved at the router level without redesign (e.g., private mempools/Flashbots Protect, on-chain TWAP guard rails, or routing through MEV-protected RPCs). Document clearly for integrators that `amountOutMin`/`amountInMax` must be computed with realistic slippage tolerance and that `deadline` should be set tightly; recommend users submit swaps via private transaction relays for large trades.

```
amounts = UniswapV2Library.getAmountsOut(factory, amountIn, path);
require(amounts[amounts.length - 1] >= amountOutMin, 'UniswapV2Router: INSUFFICIENT_OUTPUT_AMOUNT');
```

F-003

Informational

Non-fee-on-transfer-aware functions will misbehave or revert with deflationary/fee-on-transfer tokens

UniswapV2Router02.sol:212–236

DESCRIPTION

`addLiquidity`, `removeLiquidity`, `removeLiquidityETH`, `swapExactTokensForTokens`, `swapTokensForExactTokens`, `swapTokensForExactETH`, and `swapExactTokensForETH` assume the full nominal amount is received by the pair/recipient. For fee-on-transfer or rebasing tokens this assumption is false, which can cause reverts (due to `INSUFFICIENT_*_AMOUNT` checks) or incorrect output accounting. The contract does provide `...SupportingFeeOnTransferTokens` variants specifically to mitigate this, but a caller/integrator who selects the wrong function variant for such a token will experience unexpected failures or fund-handling issues.

IMPACT

Integrators or end users unaware of a token's fee-on-transfer/rebasing behavior may have transactions revert (benign) or, in edge cases with certain token implementations, receive fewer tokens than expected when using the non-supporting variants together with a mismatched `amountMin`.

RECOMMENDATION

No code change required in the router; ensure front-end integrations detect fee-on-transfer/rebasing tokens (e.g., via a pre-flight balance-delta check) and automatically route to the `SupportingFeeOnTransferTokens` function variants.

```
function removeLiquidityETH(address token, uint liquidity, uint amountTokenMin, uint
amountETHMin, address to, uint deadline)
public virtual override ensure(deadline) returns (uint amountToken, uint amountETH)
{ ... }
```

F-004

Informational

Metadata/deployment classification mismatch: contract flagged 'Upgradeable: yes' but implementation is immutable with no proxy pattern

UniswapV2Router02.sol:148–152

DESCRIPTION

The provided metadata marks the contract as upgradeable; however, the source shows `factory` and `WETH` declared `immutable` and set only in the `constructor`, with no delegatecall-based proxy, no initializer pattern, and no admin/upgrade functions. This appears to be an input/metadata labeling discrepancy rather than an on-chain characteristic.

IMPACT

None to the contract itself; flagged purely to correct the risk model, there is no upgrade authority, storage-collision risk, or unprotected-initializer risk (SWC's typical upgradeable-proxy pitfalls) applicable to this specific deployment.

RECOMMENDATION

Confirm deployment metadata/classification tooling; no code change needed.

```
address public immutable override factory;
address public immutable override WETH;

constructor(address _factory, address _WETH) public {
  factory = _factory;
  WETH = _WETH;
}
```

Gas Optimizations

Pure/view wrapper functions declared `public` instead of `external`

UniswapV2Router02.sol:392–433

`quote`, `getAmountOut`, `getAmountIn`, `getAmountsOut`, and `getAmountsIn` are never called internally within the router contract and are only invoked externally; declaring them `external` instead of `public` avoids unnecessary calldata-to-memory copying for array parameters and reduces deployed bytecode/gas slightly.

Repeated `UniswapV2Library.pairFor` computation

UniswapV2Router02.sol:248–420

Several functions (e.g., `removeLiquidityWithPermit`, `swapExactTokensForTokens`) recompute `pairFor` multiple times for the same token pair within a single call path. Cache the computed pair address in a local variable and reuse it to save redundant keccak256/address-casting operations.

Appendix - Metadata

Contract	UniswapV2Router02
Address	0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D
Chain	Ethereum
Compiler	v0.6.6+commit.6c089d02
Audit ID	portfolio-uniswap-v2-router02
Date	10/7/2026

Files analyzed

[user] UniswapV2Router02.sol

MANUAL AUDIT UPGRADE

Need a deeper audit?

\$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

[Request Manual Audit '](https://cryptohawking.com/audit/manual?ref=portfolio-uniswap-v2-router02)

<https://cryptohawking.com/audit/manual?ref=portfolio-uniswap-v2-router02>

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

<https://cryptohawking.com/services>

WhatsApp (preferred): +92 322 4274236, <https://wa.me/923224274236>